

Section 1: About Me

Goals and Objectives

Why do you want to do a GSoC project with Oppia?	I've been contributing to Oppia since last year, so I'm already familiar with its codebase, workflows, and development standards. This makes it easier for me to dive in and start contributing effectively. More importantly, I really connect with Oppia's mission of providing free education. Knowing that my work can help make learning more accessible and impactful keeps me motivated to contribute.	
What do you hope to learn/achieve during GSoC?	I want to deepen my understanding of testing and improve my debugging skills. Through this experience, I hope to become more efficient at identifying and fixing issues while writing high-quality, reliable code.	
Which Oppia teams have you collaborated with, and what have you done on those teams?	Developer Workflow Team	I have assisted both new and existing contributors in setting up the Docker environment and have also reviewed a few pull requests.
Contact information	• GitHub: https://github.com/jayam04 • Email: jayampatel04@gmail.com (Also available through Google Chat).	
Preferred method of communication	• Email at jayampatel04@gmail.com • Google Chat at jayampatel04@gmail.com for urgent but less important notifications	
Social media handles (optional) <i>This field is optional. We ask for it so that we can tag you in GSoC-related posts if your application is accepted!</i>	LinkedIn: https://linkedin.com/in/jayam04	
Which time zone will you primarily be in during the summer?	Indian Standard Time (GMT +5:30)	
(If you are a student) When are your school holidays?	I am currently in the final semester of my degree program. As part of my curriculum, I am completing an internship, which is scheduled to conclude by April 15, 2025. I may begin a new degree program in September. During the interim period, I will have ample free time and can dedicate myself to working on Oppia full-time.	
What other obligations might you need to work around	I have no obligations from the beginning of May to the end of August. However, since I may start a new degree program in September, I might need to begin preparations towards the end of August. I have taken this into	

during the summer?	account and provided further details in the next section (Planned Time Commitment).
Planned time commitment	From June 2 to mid-August, I plan to dedicate 6–7 hours per day, Monday to Friday, and 3 hours on weekends to the project. After mid-August, while I will still be available, I may need to begin preparations for a new degree program. To accommodate this, I aim to complete the majority of the work by mid-August. During the remaining period, I will still be able to commit at least 5 hours per week for tasks such as reviewing and resolving PRs. Total commitment: • Full-time availability for 11 weeks • Reduced workload for the last 4 weeks (5 hours per week)¹

Project Timeframe

Note: *Oppia will only be offering a single GSoC coding period timeframe this year, starting on **June 2**. All work for Milestone 1 must be completed and submitted by **Jul 4** for internal feedback (with any revisions due by **Jul 18**), and all work for Milestone 2 must be completed and submitted by **Aug 27** for internal feedback (with any revisions due by **Sep 10**). We will not be able to extend these deadlines.*

Coding period	• I will adhere to the above deadlines.
---------------	---

Communication Channels

Note: *The Oppia team places a high emphasis on communication, and we have found that daily contact between contributors and mentors is important for helping keep projects on track. This is why we ask that contributors send short daily updates to their mentors explaining what they have done, where they are stuck, and what they plan to do next.*

I can commit to sending daily updates to my mentor by email, each day I work during the GSoC period.	Yes, I commit to send updates to my mentor daily by email.
In addition to the above: how often, and through which channel(s), do you plan on communicating with your mentor?	In addition to the above updates, I plan to have a detailed conversation with my mentor on Google Chat or Weekly meeting on a weekly basis to discuss progress, tasks due, and future plan.

¹Starting in early September, I will be enrolled in a degree program with a course load of 8 credit hours. Based on the standard estimate of 3 hours of study per credit hour, this will require a weekly commitment of approximately 24 hours (around 4.8 hours per day). Based on my previous scheduling experience, I believe I can still allocate 2 hours per day to Oppia, totaling 12 hours per week (excluding Sundays). **While I've conservatively estimated only 5 hours per week in my official schedule, I anticipate being able to contribute more.**

Contributor Briefing

Note: A couple of days after the selection results are announced, there will be a mandatory contributor briefing for accepted contributors. Please confirm whether you will be able to attend the briefing.

If accepted, I will be able to attend the contributor briefing at 3 pm UTC on May 10.	• Yes
---	---

Section 2: About My Project

Project Details

Project title (should match the one on the Project Ideas list)	Acceptance tests
Project size (should match the options on the Project Ideas list)	Large (~350 hours)
Why did you choose this project?	I chose this project because it will help me enhance my testing and debugging skills. Currently, I have only a basic understanding of software testing. By working on this project, I aim to strengthen my ability to write robust tests and improve my debugging skills by resolving flaky tests in the CI pipeline.

Required Skills

GENERAL	
Note: While drafting this proposal, I have pending PRs and approvals due to a busy schedule of mine and mentors (Christie) managing CUJs. Please check the following link to see the latest contributions that demonstrate my skills required for this proposal. Web Link	
I can figure out the root cause of an issue, and communicate it effectively using a debugging doc .	1. 2025-02-23 Debugging Doc for Issue #20869: [Flake]: Err... "Error: Goals Section is Empty" 2. Debugging Doc: [Docker] Failing E2E Test (fileUploadFeat... "Button Value expected to be false instead of true."
I can debug and fix CI failures / flakes.	1. Part of PR#19962 (commit 27f7e486): I debugged that flake was caused due to short length of audio file (ie. it was the reason the audio player stopped playing audio and the test failed).

I can make changes to GitHub Actions workflows.	1. PR#21775 : Add video recording feature for acceptance test in local and CI.
WEB	
I can write Python code with unit tests.	N/A
I can write TS + Angular code with unit tests.	None covered by Karma tests.
I can write or modify acceptance tests.	1. PR#22331 : (Draft) Implement CUJ 11.9 Topic manager can't add explorations unsupported by mobile devices. While it's not merged, the acceptance test is already approved by Christie (mentor).

Oppia Community References

Who you worked with	Context in which you worked with them
Sean Lip (@seanlip)	Sean reviewed many of my PRs and has also guided me in resolving flakes and PR reviews.
Om (@omgupta-iitk)	Collaborated on debugging a flake.

Section 3: Proposal Details

Note: This section is largely adapted from [Oppia TDD Template \(Dec 2024\) – PLEASE MAKE A COPY](#). We **strongly recommend** reading that doc, since it has detailed explanations of how to fill out the various sections. It also contains links to sample TDDs, so that you can see what others have done in the past.

Problem Statement

Target Audience	Developers and Maintainers of Oppia Codebase
Core User Need ²	While writing new code in a small codebase is relatively straightforward, introducing changes in a large codebase often leads to bugs and conflicts. To prevent new code from breaking existing functionality, various testing techniques are employed. One such technique is acceptance testing, which ensures that the website behaves and functions as expected. In Oppia, we have several Critical User Journeys (CUJs)—key workflows that most end-users are expected to follow. Our goal is to develop acceptance tests that comprehensively cover all CUJs, ensuring that the website remains stable and

² Source: Project Ideas 3.1 Acceptance Tests (Oppia)
<https://github.com/oppia/oppia/wiki/Google-Summer-of-Code-2025#31-acceptance-tests>

	bug-free across essential workflows. Currently, we use WebdriverIO for end-to-end (e2e) testing. Our goal is to ensure that all e2e tests are covered by Critical User Journeys (CUJs) in acceptance tests. Once an e2e test is fully covered by an acceptance test, we need to remove the corresponding WebdriverIO test to maintain a cleaner and more efficient test suite. We also need to ensure all the utility functions are tightened. This means there should be proper pre-checks and post-checks associated with each function, with proper logging. This ensures high standards which would help in faster debugging of flakes and other errors.
--	---

Section 3.1: WHAT

Critical User Journeys are common journeys through which many end users go through. While assuring quality of software it's hard to go through each journey manually and check if the software behaves in an unexpected way (bugs). Thus, we use acceptance tests which checks different CUJs and makes sure there are no bugs in the software.

Acceptance tests help us to catch critical bugs in the software³. At Oppia, we have many different end users having different roles such as learner, curriculum admin, topic manager, and more. We need to ensure that acceptance tests cover CUJs of users with different roles. We have a Critical User Journey (v2) ([link](#)) doc which lists CUJs with their current status - Implemented, Partially Covered, Incomplete.

At Oppia, we also have end-to-end (E2E) tests. E2E tests are also used to test functionality of our product (Oppia Webpages), but unlike acceptance tests, these are categorized by pages rather than users. To have one consistent testing system, we need to make sure all of the different tests checked by E2E tests are migrated to acceptance tests. Once all of the functionality tests by E2E test are also implemented in acceptance tests, we need to remove that specific E2E test⁴.

Flakes

When tests are executed, browsers are controlled by tests. Many times, the behaviour of browsers are non-deterministic and causes flakes⁵. While there are many reasons for flakes such as race conditions, leaked state, network/third-party dependency, and randomness, race conditions are most common amongst them⁶. Race conditions can further occur due to different reasons, such as opening a browser, page, saving data in datastore, etc.

When implementing the CUJs using acceptance tests, we need to ensure tests are least flaky. We also need to make sure that whenever flaky tests occur, developers can easily find the root cause and fix the flaky test.

³ Source: Acceptance Tests (Oppia) <https://github.com/oppia/oppia/wiki/Acceptance-Tests>

⁴ Source: End-to-End Tests (Oppia) <https://github.com/oppia/oppia/wiki/End-to-End-Tests>

⁵ Source: End-to-End Tests (Oppia) <https://github.com/oppia/oppia/wiki/End-to-End-Tests>

⁶ Source: What causes flaky tests (Code with Jason) <https://www.codewithjason.com/what-causes-flaky-tests/>

Additional Project Related Work

I plan to do the following three additional works, as many of CUJ are claimed and written. These are ordered based on priority.

1. Organizing tests properly by CUJs

Work with the QA team to organize tests properly by CUJs. It includes tasks such as adding new CUJs not covered by current CUJ doc, removing duplicate CUJs, etc.

2. Better Infrastructure for Debugging (Datastore Logs)

In addition to writing acceptance tests, this project should also add features to show more logs. More specifically, developers would love it when they can see changes in datastore objects while debugging flakes. Issue [#22185](#) is filed and has subtask - Datastore logs.

More in [3.2 How / Datastore Logs](#) section.

3. Fix some of the existing flakes.

Fix some of the existing flakes. While I do plan to fix existing flakes, the number of flakes will depend on time remaining. Thus, this will be the last task of Milestone 2.

Key User Stories and Tasks that will be implemented

#	Title	User Story Description (role, goal, motivation) <i>"As a ..., I need ..., so that"</i>	List of tasks needed to achieve the goal (this is the "User Journey")
1	Developer needs to verify his changes are good.	As a developer I need CI tests that check CUJs using acceptance testing, so that I am sure that my changes don't break currently working things in project	Developer pulls code from the Oppia repository.
			Developer makes required changes to resolve any specific issue or implement any feature.
			Developer creates a PR in the main Oppia repository.
			Developer checks for Acceptance tests status in full-stack tests in CI.
			Developer updates his code, if any acceptance test fails
2	Code reviewers needs to easily identify breaking changes	As a code reviewer, I need want CUJs to be covered by acceptance testing, so that whenever I need to review a new PR, I can easily check that no current CUJ are	A developer asks me to review their code changes.
			I need to make sure, new code doesn't introduce new bugs.
			I expect CI to automatically test critical user journeys (CUJs) using acceptance tests.
			When all acceptance tests pass in PR, I am assured that no

		affected by new code	critical bugs (bugs that hinder CUJ) are introduced by the code changes.
			If there are failing tests, I will check if they are caused by new code and ask the developer to resolve them.
			If test failure is due to flake, I will file a new issue stating the flake and test logs.
3	Developer want to quick find root cause of flake	As a developer, I want an easier process in debugging the flakes.	A new flake is reported.
			I check CI logs.
			Using pre/post check logs, I am able to approximate the point where flake occurs.
			If pre-check passes, but post check fails; it's much likely that flake occurred in the current state.
			If the pre-check fails, it ensures the test doesn't move forward and I don't need to check extra steps.
			Additionally, if screenshots are taken at failure point, it would make the debugging process easier.

Testing Plan

This project doesn't introduce any new features or bug resolutions (except for flakes). Thus, no CUJs except listed in [Critical User Journeys](#) need to be implemented⁷.

However, as we need to make sure tests are not flaky, we will use rigorous CI runs as explained in ["Applying a fix"](#). In brief, each acceptance test will be tested for flakiness before merging it to the *develop* branch.

Technical Requirements

Additions/Changes to the Web Server Interface

N/A

New/Changed Calls by the Web Frontend /Android Client to the Web Server Interface

N/A

Web Server Storage Model Additions/Changes

N/A

⁷ Though CUJs added by the QA team will be covered.

UI Screens/Components

None, except for adding classes and / or IDs to HTML components used in new acceptance tests.

Data Handling and Privacy

N/A

Connection to Existing Work

N/A

Other Requirements

N/A

Section 3.2: HOW⁸

Existing Status Quo

Currently, we already have infrastructure for acceptance tests. This includes acceptance testing in local machines, CI checks in PRs and the 'develop' branch.

Existing Infrastructure

At a high level, we already have python scripts and GitHub actions that help us run acceptance tests. We use `python -m scripts.run_acceptance_tests -suite=<SUITE_NAME>` to run tests locally. Acceptance tests in CI are covered by `full_stack_tests.yml`.

On a lower level, we are using Jest for acceptance tests with Puppeteer to control browsers. Jest is known for its simplicity, speed, and built-in features, making it one of the most popular choices for testing JavaScript applications⁹¹⁰. Puppeteer is a JavaScript library which provides a high-level API to control Chrome or Firefox over the DevTools Protocol or WebDriver BiDi¹¹.

File Structure

All of the acceptance tests are stored in `oppia/core/tests/puppeteer-acceptance-tests` with the following folder structure¹². This puppeteer-acceptance-test folder is further divided into 4 main folders - data, functions, specs, and utilities. specs folders hosts all of the test files, while utilities folder has utilities

⁸ Note: Page limit of How section exceeds only due to huge table listing all CUJs which must be covered by this project including ones that are added afterwards.

⁹ Source: Testing with Jest (GeeksForGeeks) <https://www.geeksforgeeks.org/testing-with-jest/>

¹⁰ Jest's website is also so simple that I need to refer to third-party sources about its basic intro. Though its documentation is full on its own.

¹¹ Source: Puppeteer (Puppeteer) <https://pptr.dev/>

¹² Source: Acceptance Tests (Oppia) <https://github.com/oppia/oppia/wiki/Acceptance-Tests>

functions used in tests. This creates a strong separation between logic / control and flow of the test. Flow of every test is controlled by *spec* files in *specs* folder, while logic must be implemented in *utilities* folder.

```
oppia/core/tests/
├── puppeteer-acceptance-tests
│   ├── data
│   │   ├── blog-post-thumbnail.svg
│   │   └── ...
│   ├── functions
│   │   └── is-element-clickable.ts
│   ├── specs
│   │   ├── blog-admin
│   │   │   ├── assign-role-to-users-and-change-tag-properties.spec.ts
│   │   │   └── ...
│   │   ├── curriculum-admin
│   │   │   ├── create-publish-unpublish-and-delete-topic-and-skill.spec.ts
│   │   │   └── ...
│   │   ├── topic-manager
│   │   │   ├── create-and-delete-subtopic-and-story.spec.ts
│   │   │   └── ...
│   └── utilities
│       ├── common
│       │   ├── puppeteer-utils.ts
│       │   ├── show-message.ts
│       │   ├── test-constants.ts
│       │   ├── console-report.ts
│       │   └── user-factory.ts
│       └── user
│           ├── blog-admin.ts
│           ├── blog-post-editor.ts
│           └── ...
```

Users Files

As acceptance tests are divided by user journeys, logics are divided by users - *LoggedOutUser*, *LoggedInUser*, *TopicManager*, etc. These all different user types are derived from *BaseUser*. We need to ensure that common logics (such as *navigateToExploration*) are implemented in *BaseUser*, ensuring consistency across all users.

Solution Overview

Knowing CUJs to test

Critical User Journeys are covered by the QA Team. Currently, all of the CUJs are listed in [Critical User Journey v2 Doc](#). We also have [Web QA Test Matrix Sheet](#) listing tests by user with detailed info, such as test steps, and test expectations. While the above doc and sheet consists of most of the CUJs, the QA team is working on tidying. This will not only change test numbers of some of the tests, but also add

more CUJs. **This project not only includes writing CUJs covered by CUJ v2 doc but also all new CUJs added until end of GSoC contribution period.**

Please refer to [Appendix A](#) which consists of all CUJs currently planned to complete by this project. As we also have *Web QA Test Matrix Sheet*, which is not synced with *CUJ v2 Doc*, [Appendix C \(CUJ v2 Doc and Web QA Matrix Sync\)](#) consists of *Web QA Test Matrix Sheet* by *CUJ v2 Doc*. Appendix C also gives numbers to CUJs that are not covered by *CUJ v2 Doc*.

File Structure

For the file structure, we plan to stick with the current structure at a higher level that all of the logical code goes to */utilities* and test (flow) code goes to */specs*. Both of these folders are divided further. */utilities* contains */common* and */user* for common and user specific logic. */specs* folder is divided by each user.

On a lower level, we need to ensure test files names in */spec/** matches with [CUJ v3 Doc](#) rather than [CUJ v2 Doc](#) used currently. Each user folder should contain all and only test files matching CUJ v3 Doc.

All of the CUJs in one blue section in CUJ v3 doc goes into one acceptance test. Thus, we will use the heading of the blue section as filename and each CUJs will have it's own it() block in the test.

In the following example:

5. First-time (logged-out) learner (FL)

Goal		Steps	Expectations
FL.CP. Learner discovers the website and navigates to the Math Classroom page			1
1.1	Find the list of subjects I can learn	Visit the splash page.	The splash page displays correctly.
		Click "Explore Oppia classrooms"	
1.2	Start learning from the first topic	Click "New to math?"	The learner is taken to the Place Values topic.
		Exit the topic by clicking on ...	The learner is taken back to the Math Classroom page.
1.3	Figure out which topic would be best for me to learn		

Filename:

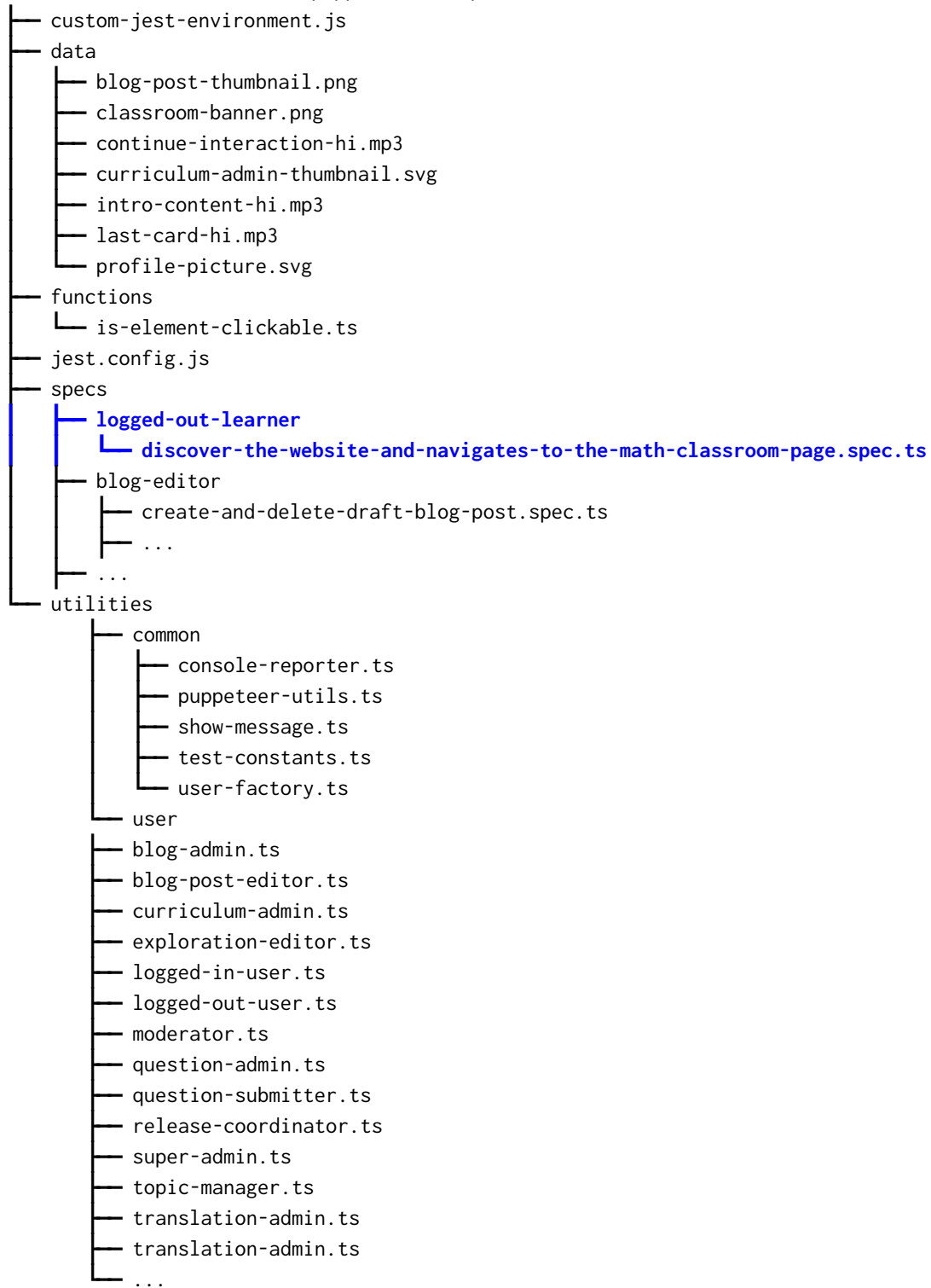
[/specs/logged-out-learner/discovers-the-website-and-navigates-to-the-math-classroom-page.spec.ts](#)

File Structure:

```
describe('First-time Learner', () {  
  beforeAll()  
  it('should be able to find the list of subjects I can learn')  
  it('should be able to start learning from the first topic')  
  it('should be able to figure out which topic would be best for me to learn')
```

```
    afterAll()
  })
```

Tree structure in `/core/tests/puppeteer-acceptance-tests` folder



Writing a test

All of the acceptance tests covering CUJs need to be written in `specs/<user>/` folder. Names of test files should explain what test covers. Every test consists of 3 parts - *beforeAll*, *it*, and *afterAll*. *Before all* contains all of the steps which are required to test a specific CUJs but are not covered by CUJ. For example, if our CUJ covers “logged out user to complete an exploration”, creating that specific exploration will be implemented in *beforeAll*. In *it()*, we cover CUJ tested by that specific test. Similarly, *afterAll* closes all browser sessions and other work required by the test is completed.

All of the logical code must be implemented in the *utilities* folder. Each function implemented should perform a specific task with proper pre- and post-checks. For instance, after publishing an exploration we can add a check that the publish button is disabled. This ensures that the test doesn't move forward if there is any issue while publishing the exploration. Similarly, we can add pre-check in methods such as `navigateToNextCard` where we check current card content. Following utility function created in [PR#22331](#) demonstrates usage of pre-checks and post-checks.

```
/**
 * This function creates simple Programming Exploration.
 * Starts at the new Exploration Editor Page.
 * Ends at same page, after adding programming interaction and saving the
 * draft.
 */
async createSimpleProgrammingExploration(): Promise<string | null> {
  // Check if element to add interaction is visible (pre-check)
  await this.page.waitForSelector(stateEditSelector, {
    visible: true,
  });

  // Logic that creates a simple programming exploration.

  // Check if publish button is disabled (post-check)
  const publishButton = await this.page.$(saveChangesButton);
  const isDisabled = await this.page.evaluate(
    el => el.disabled,
    publishButton
  );

  if (isDisabled) {
    showMessage('Publish Button is disabled, as expected');
  } else {
    showMessage(
      'Publish Button is enabled and clickable, expected to be disabled'
    );
    throw new Error('Publish Button is enabled and clickable');
  }

  return explorationId;
}
```

Thus, pre- and post- checks can contain many wait conditions and checks such as (but not limited to):

- Wait for the network to stabilize.
- Check for an element to be in a specific state.
- Wait for elements to appear / disappear (such as toast).

Additionally, we are using Typescript, functions should have proper type linting. We also need to make sure the function throws appropriate errors whenever needed. We will also need to implement some functions for embedded player CUJs. We can use puppeteer frame objects for the same.

`puppeteer.Frame` supports many methods that `puppeteer.Page` supports.

Flakes

A flaky test is a test that passes sometimes and fails sometimes, even though no code has changed¹³. Flaky tests are currently very common in acceptance tests. We need to ensure that new CUJs implemented are flake free. To achieve this, an important thing is to add tightening in user functions. This will help us debug flakes easily by finding the root cause quickly. For example, if a function has loose logic, it may perform unwanted things which won't be caught until the next few steps. These types of flakes will be tedious to fix. Thus, tightening the function is very important.

Debugging Docs is also a great feature to use while debugging a flaky test. At Oppia, we have a debugging doc [template](#), this can be used to test hypotheses, and to get external help from other contributors.¹⁴ Another useful feature we can use to debug flakes in recorded videos in CI. I created PR [#21775](#) which adds a video recording feature to CI, recording acceptance tests that fail in CI.

Actually Debugging a Flake

Fixing a flake involves 3 major steps: Reproduction, Diagnosis, and Fix¹⁵.

Reproduction

Reproduction of flakes can be tricky, especially when the CI environment has huge differences from the local environment like in Oppia. However, a CI can be simulated locally by running an acceptance test using only one or two CPU cores. To achieve this, we can use tools such as `cpulimit` and `taskset`. I have used a taskset in the past when debugging a flaky test. Taskset has a simple command `taskset -c <N> command`, where N is the number of CPU cores to limit. Taskset also imposes this limit to subprocesses thus ensuring all subprocesses started by python script also have limited CPU resources. Following steps helps us install taskset and use binary search to determine the number of CPUs to use:

1. Install Taskset¹⁶
2. `$ sudo apt-get update`
3. `$ sudo apt-get install util-linux`
4. Run acceptance test using all GPUs
5. Check if the test fails at line before failure point in CI or after failure point.
6. Based on the above check, use binary search to increase / decrease the number of CPUs for testing.

¹³ Source: What causes flaky tests (Code with Jason) <https://www.codewithjason.com/what-causes-flaky-tests/>

¹⁴ Source: Debugging Docs (Oppia Wiki) <https://github.com/oppia/oppia/wiki/Debugging-Docs>

¹⁵ Source: How I fix flaky tests (Code with Jason) <https://www.codewithjason.com/how-i-fix-flaky-tests/>

¹⁶ Source: Linux Taskset Command (I/O Flood) <https://ioflood.com/blog/install-taskset-command-linux/>

On the other hand, reproduction of flake on CI is a pretty straightforward process, where we modify `full_stack_tests.yml` by temporarily hardcoding acceptance tests in command. More specifically, we updating `python -m scripts.run_acceptance_tests --skip-build --suite=${{ matrix.suite.name }}` to `python -m scripts.run_acceptance_tests --skip-build --suite=<FLAKY_TEST_SUITE>--prod_env.`

Diagnosis

To diagnose a test, the first thing to do is check failure logs. This helps us check where the flake is occurring. We can then check recorded video and detailed logs to check what is wrong. For example, a topic should be marked as completed, but it isn't. Then, the next step would be to check potential causes which can cause this error. For example, the topic was actually updated but was overridden by another method, the topic was not updated in the first place, or the topic did update but wasn't fetched correctly.

Debugging Docs can be useful in this stage, as we can get help from other contributors and test our different hypotheses. At Oppia, this is the most preferred way to debug a flake. We first create a debugging with some hypothesis, then we post this debugging doc in the GitHub Discussions page, where everyone can collaborate on debugging the flake.

Applying a Fix

While this step seems pretty straight forward, one of the important things is to ensure our fix actually works. We can run the test in the CI environment of our fork multiple times to ensure that our solution actually fixes the flake. To achieve the same, we can create a temporary workflow from `full_stack_tests.yml`, but only with acceptance tests and using the number of runs in strategy and matrix provided by GitHub workflows. We also need to update the `acceptance_test` job to ensure video is recorded in every test.

Steps:

1. Create a temporary workflow file that runs given acceptance for a specific number of times.
2. Check for flakes
3. Apply Fix
4. Repeat step 2-3 until flakes are found
5. Remove the temporary workflow file
6. Create a PR

Workflow file (Draft)

name: Full-stack tests

permissions: read-all

on:

pull_request:

branches:

- develop
- release-*

workflow_dispatch:

concurrency:

group: \${{ github.workflow }}-\${{ github.event.pull_request.number || github.run_id }}

cancel-in-progress: true

```

jobs:
  e2e_and_acceptance_coverage:
    ...
  build:
    ...
  acceptance_test_with_python_installation:
    needs: [build]
    runs-on: ubuntu-22.04
    strategy:
      matrix:
        attempt: ${{ range(1, 101) }}
        fail-fast: false
    name: Acceptance (Attempt ${{ matrix.attempt }})
    steps:
      - name: Checkout repository so that local actions can be used
        uses: actions/checkout@v4
      - name: Merge develop and set up dependencies
        uses: ../github/actions/merge-develop-and-set-up-dependencies
      - name: Generate build files
        uses: ../github/actions/generate-build-files

      - name: Run Desktop Acceptance Test Attempt ${{ matrix.attempt }}
        run: >
          VIDEO_RECORDING_IS_ENABLED=1
          xvfb-run -a --server-args="-screen 0, 1920x1080x24"
          python -m scripts.run_acceptance_tests
          --skip-build --suite=<TEST_SUITE_TO_CHECK> --prod_env
      - name: Uploading Desktop test recordings as Artifacts
        ...
      - name: Uploading webpack bundles as an artifact
        ...
      - name: Uploading diff screenshots as an artifact
        ...
      - name: Report failure if failed on oppia/oppia develop branch
        ...

```

Removing E2E tests covered by Acceptance Tests

As stated above, E2E tests need to be removed once it is fully covered by acceptance. As E2E tests are based on pages / features while acceptance tests are based on user journeys, multiple acceptance tests cover a E2E test collectively. [Appendix B](#) consists of a table that shows coverage of E2E tests by CUJs.

Datastore Logs

With this project, we want to have a feature using which developers can check datastore changes for a specific or every model. It will be done using `--watch-datastore-model` argument while running `python -m scripts.run_acceptance_tests`. We can use the currently available datastore module from google.cloud.

Fetch Datastore Changes (Draft)

```
client = datastore.Client()
```

```
def get_recent_changes(last_checked):
    query = client.query(kind='YourKind')
    query.add_filter('last_updated', '>', last_checked)
    results = list(query.fetch())
    return results
```

To implement this, we also need to ensure the following things:

- Abstract required functionality in core...datastore, as we do not use google.cloud modules directly.
- Ensure logging starts / stops correctly with tests. It should have no impact on test status (success/fail).

Third-Party Libraries

N/A

“Service” Dependencies

N/A

Key High-Level and Architectural Decisions

N/A

Key Objectives for each Milestone¹⁷

For each milestone, describe its “key objective”, i.e. what the end-user will be able to do once all the code for that milestone has been fully merged.

*By default, your “key objective(s) for each milestone” should match the one given in the [project idea description](#). If any changes are needed, please create a **public thread** in the relevant category on [GitHub Discussions](#), and discuss this with the org admins **before** submitting your proposal.*

Milestone	Key Objective
Milestone 1	Tighten all existing page utility functions in <code>core/tests/puppeteer-acceptance-tests</code> to have appropriate pre/post checks. Complete all acceptance tests for exploration creators, logged-out users, logged-in users, curriculum admin, and blog admins (as specified in #21646), and ensure that they run on all PRs by adding them to the “acceptance test” GitHub workflow. Remove any existing webdriverio tests whose functionality is fully covered by the new acceptance tests.
Milestone 2	Complete all other remaining acceptance tests (as specified in #21646), and ensure that they run on all PRs by adding them to the “acceptance test” GitHub

¹⁷ Source: Project Ideas 3.1 Acceptance Tests (Oppia)
<https://github.com/oppia/oppia/wiki/Google-Summer-of-Code-2025#31-acceptance-tests>

	workflow. Remove any existing webdriverio tests whose functionality is fully covered by the new acceptance tests. If any webdriverio tests remain after this step, translate them into CUJs and work with the QA team to make them part of the CUJ document, then implement the corresponding acceptance tests. Finally, remove the old webdriverio and e2e test framework completely.
--	---

Implementation Approach

CUJ are numbered based on user and from basic CUJ to CUJ with more functions (within each user). Thus, the implementation approach is pretty straightforward where we start implementing CUJs one after another in chronological order.

As I plan to start implementation in CBP, there can be some possibility when few CUJs are written out of chronological order.

Storage Model Layer Changes

N/A

Storage Model Migrations

N/A

Domain Objects

N/A

User Flows (Controllers and Services)

N/A

Web frontend changes

N/A

Documentation changes

N/A

Metrics Plan

N/A

Implementation Plan

Milestone Table (include both PRs and other actions that need to be taken prior to launch)

This project already has all the infrastructure required. Given this and a possibility of a somewhat busy schedule of mine after August 15, 2025, I plan to start coding from **May 12, 2025** itself. Additionally, in this timeline I have given 10%-15% more time to each subtask. I plan to complete all subtasks within their timeframe and use "Complete Backlogs" subtasks to move ahead and fix flakes.

I have added buffer time to complete Milestone 1 from July 2 to July 9. Additionally, there are two "Complete backlogs, if any." subtasks of 3 days each. This gives extra time of almost 2 weeks for clearing backlogs.

I also acknowledge that CBP is from May 8 to June 1 and will be used in organizing tests. I plan to help organize tests and simultaneously work on a GSoc project as per the timeline given below.

(*) Partially covered: CUJ is already partially covered.

(S) Short: CUJ is straightforward most of utilities functions are already written

(M) Medium: Some utility functions need to be written.

(L) Long: Most utility functions need to be written.

Milestone 1

Key objective for this milestone:

Tighten all existing page utility functions in `core/tests/puppeteer-acceptance-tests` to have appropriate pre/post checks. Complete all acceptance tests for **exploration creators, logged-out users, logged-in users, curriculum admin, and blog admins**¹⁸ (as specified in [#21646](#) and [CUJ v3 Doc](#)), and ensure that they run on all PRs by adding them to the "acceptance test" GitHub workflow. Remove any existing webdriverio tests whose functionality is fully covered by the new acceptance tests.

No.	Description of PR / action	Prerequisite steps	Target dates (2025)	
			PR creation	PR merge
A	Complete Topic Manager, Voiceover Admin , and Voiceover Artist CUJs		May 25	May 28
B	Write CUJs of the Practice Question Submitter in CUJ v3 Doc.		May 28	May 31
C	Write CUJs of the Practice Question Reviewer in CUJ v3 Doc.		May 31	June 2

¹⁸ Milestone 1 differs slightly from Oppia Wiki as many of CUJs, which were to be covered by Milestone 1, are already completed. This will help us get more time at the end which can be used to add more features to testing.

1	Tighten Utility functions by adding pre/post checks of <i>logged-in-user.ts</i>		June 3	June 6
2	Implement acceptance tests for selected Critical User Journeys (CUJs), including: • Logged-In User (#9) with codes – LI.OP, LI.EP • Logged-Out User (playing an exploration) (#6) with codes – EL.SL, EL.LP, EL.FB, EL.VO, EL.HC, EL.CP, EL.CL • Logged-Out Learner (playing an embedded lesson) (#7) with codes – EL.EE (Refer to CUJs #9, #6, and #7 in the CUJv3 document.)		June 7	June 9
3	Tighten Utility functions by adding pre/post checks of <i>logged-out-user.ts</i>		June 10	June 13
4	Tighten Utility functions by adding pre/post checks of <i>exploration-editor.ts</i> and <i>curriculum-admin.ts</i>		June 11	June 15
5	Tighten Utility functions by adding pre/post checks of other users • <i>blog-admin.ts</i> • <i>blog-post-editor.ts</i> • <i>moderator.ts</i> • <i>question-admin.ts</i> • <i>question-submitter.ts</i> • <i>release-coordinator.ts</i> • <i>super-admin.ts</i> • <i>topic-manager.ts</i> • <i>translation-admin.ts</i> • <i>voiceover-admin.ts</i>		June 14	June 21
I will be on break from June 16 - June 20				
6	Implement Acceptance tests for CUJs of Logged-In User (#9, and #10 in CUJs v3 doc) E2E tests to be removed: subscriptionsFlow		June 22	June 28
7	Implement Acceptance tests for CUJs of First-time (logged-out) learner, Learner (logged-out) playing an exploration, Learner (logged-out) plays an embedded exploration, and Learner (logged-out) visits the community library (#5, #6, #7, and #8 in CUJ v3 Doc) E2E tests to be removed: accessibility, blog, collections, embedding, explorationFeedbackTab, learnerFlow, and		June 27	July 1

	profileFeatures			
8	Implement Acceptance tests for CUJs of Exploration Creator (#11 in CUJs v3 doc) E2E tests to be removed: fileUploadExtensions, creatorDashboard, explorationHistoryTab, explorationStatisticsTab, publicationAndLibrary		July 4	July 8
9	Implement Acceptance tests for CUJs of Interested Partner Organization, Interested Donor, Interested Volunteer, and Interested Parent (#1, #2, #3, and #4 in CUJ v3 Doc)		July 5	July 8
10	Flake #22448 (contributorDashboard)		July 8	July 10
11	Flake #22448 (explorationTranslationTab)		July 10	July 12
12	Flake #22448 (fileUploadFeatures)		July 12	July 14

Milestone 2

Key objective for this milestone:

Complete all other remaining acceptance tests (as specified in [CUJ v3 Doc](#)), and ensure that they run on all PRs by adding them to the "acceptance test" GitHub workflow. Remove any existing webdriverio tests whose functionality is fully covered by the new acceptance tests. If any webdriverio tests remain after this step, translate them into CUJs and work with the QA team to make them part of the CUJ document, then implement the corresponding acceptance tests. Finally, remove the old webdriverio and e2e test framework completely.

No.	Description of PR / action	Prerequisite steps	Target dates (2025)	
			PR creation	PR merge
1	Implement Acceptance tests for CUJs of Topic Manager (#12 in CUJs v3 doc) E2E tests to be removed: topicsAndSkillsDashboard, topicAndStoryEditor, skillEditor, profileMenuFlow, and topicAndStoryEditorFileUploadFeatures		July 19	July 24
2	Implement Acceptance tests for CUJs of Curriculum Admin (#13 in CUJs v3 doc)		July 22	July 27
3	Time period to implement slipped acceptance tests, update naming and minor changes required for old tests of Logged-Out User and Logged-In User (#1-#10 in CUJ v3 Doc)		July 24	August 2

4	Implement Acceptance tests for CUJs of Translation Submitter, Translation Reviewer, Practice Question Submitter, Practice Question Reviewer, Translation Coordinator, and Practice Question Coordinator (#14, #15, #16, #16, #17, #18, and #19 in CUJs v3 doc)		August 1	August 6
5	Time period to implement slipped acceptance tests, update naming and minor changes required for old tests for users from #11-#19 in CUJ v3 Doc		August 3	
6	Implement Acceptance tests for CUJs of Blog Post Writer, and Blog Admin (#22, and #23 in CUJ v3 Doc)		August 8	August 12
7	Implement Acceptance tests for CUJs of Release Coordinator, Site Moderator, and Super Admin (#24, #25, and #26 in CUJ v3 Doc)		August 16	August 20
8	Implement Acceptance tests for CUJs of Voiceover Admin, and Voiceover Submitter (#20, and #21 in CUJ v3 Doc)		August 18	August 22
9	Remove Webdriverio framework		August 19	August 22
10	Close all E2E / Webdriverio Issues.		August 19	August 22

Feature Flags

N/A. This project doesn't introduce any new changes.

Future Work

Implement acceptance tests in Docker. Currently, acceptance tests in Docker still use Jest / Puppeteer in a non-docker (native system). Implementing acceptance tests in docker using selenium will create a more consistent environment in CI and local machines.

Note: Currently, we have stopped supporting Docker. Additionally, Docker setup is a bit unstable, especially on low end machines, thus this future work is blocked by resuming Docker support (at least in beta) and stable docker setup.

References

1. ChatGPT (chatgpt.com) – Utilized for **paraphrasing text** to enhance readability and cohesion. **No content** in this document was generated by ChatGPT; it was solely used for improving **linguistic**

clarity. Additionally, **no part** of the document after 3.1 What / Core User Needs (in the table) is paraphrased by any AI. No AI is used except the chat in the following link.

<https://chatgpt.com/share/67f4c6f1-cd98-8006-9b9a-bfa888fe5adf>

2. Project Ideas 3.1 Acceptance Tests (Oppia)
<https://github.com/oppia/oppia/wiki/Google-Summer-of-Code-2025#31-acceptance-tests>
3. Acceptance Tests (Oppia)
<https://github.com/oppia/oppia/wiki/Acceptance-Tests>
4. End-to-End Tests (Oppia)
<https://github.com/oppia/oppia/wiki/End-to-End-Tests>
5. What causes flaky tests (Code with Jason)
<https://www.codewithjason.com/what-causes-flaky-tests/>
6. Testing with Jest (GeeksForGeeks)
<https://www.geeksforgeeks.org/testing-with-jest/>
7. How I fix flaky tests (Code with Jason)
<https://www.codewithjason.com/how-i-fix-flaky-tests/>
8. Linux Taskset Command (I/O Flood)
<https://ioflood.com/blog/install-taskset-command-linux/>

Appendix A

(CUJs to be covered by this project)¹⁹

Colors Used in Appendix Table:

 Green - Already covered

 Orange - Partially covered

 Red - Needs to be covered

 Purple - Taken up by some contributor and is expected to be completed. Note: These tests still fall under this project if not completed by the currently assigned developer and must be completed (by me). They are marked purple only to plan this project more efficiently.

CUJ v2 Doc CUJ No. / [[Given CUJ No. from Appendix C]]	Critical User Journey
1. Logged-out learner (Static pages)	
1.13	User can: • Navigate to the footer of the page where the newsletter subscription field is located. • Enter their email address into the respective field and click on the "Subscribe" button to submit their email address for newsletter subscription. • Upon successful subscription, view a "Thanks for subscribing" modal and within this modal, find and click on the "See our video" button to verify its functionality. • Similarly, within the same modal, find and click on the "Read our blog" button to confirm its functionality.
1.15	On the Blog page, user should be able to: • View all blogs, with the most recent ones appearing at the top, along with tags associated with each blog. • Search for blogs based on keywords and tags. • Utilise the paginator to navigate to subsequent pages of blogs. • Reduce the browser dimensions to simulate a small mobile screen and make sure no overlapping of buttons
[1.20]	Go to Browse the Library link

¹⁹ Source: Critical User Journey v2 (Oppia) and Web QA Test Matrix (Oppia)

https://docs.google.com/document/d/1s3MG2MVh_7m7B0wllZb7sAc0yUdY0zq7a1JEftwYBjl/edit?tab=t.0#heading=h.gs2e2lh85so7

https://docs.google.com/spreadsheets/d/1O8EHISAGrG0yoNUBz9E4DlwKNS8Rfsv_ffC4k1WK5jc/edit?gid=1807800085#gid=1807800085

[1.21]	<i>logged-out-user/click-all-links-in-oppia-footer.spec.ts</i> (Covered) 1. Go to Privacy Policy link in the footer. 2. Scroll down and Click on 'https://www.oppia.org' link 3. Click on the back button in the browser and click on 'http://www.allaboutcookies.org/manage-cookies/index.html' 4. Click on the back button in the browser and click on 'https://www.google.com/policies/privacy/partners/' (Uncovered) 5. Click on the back button in the browser and click on 'https://tools.google.com/dlpage/gaoptout'
2. Logged-out learner (Non-static pages)	
2.5	A user can start and play through a “practice questions” session from the practice tab.
2.6	A user can visit the Fractions landing page and start playing the Fractions story.
2.6.1	• The User can go to the Financial literacy page and should be able to enter decimal (.) in the answer. • You should not be able to add a duplicate unit in number with unit interaction Note:you can create your new exploration and test this or check this url. https://www.oppiatestserver.org/explore/EyJ180bShWrI
2.9	From the Community library, user can: • Visit the collection player by selecting one of the collections. • See the “collection story path” diagram, can see info about individual exploration via hovering onto the explorations in the diagram. • Share the collection. • Play any exploration in the collection.
2.10	<i>play-through-lesson-while-getting-feedback-and-hints.spec.ts</i> A user can • play through different interactions in an exploration. • view all the different rich-text components properly. They can do this even after the exploration has loaded, while they are offline. • View Oppia's feedback whenever any interaction is used. • Navigate to previously played cards if needed. • Use concept cards and hints wherever provided. • View all the previous responses to any question. • Confirm that they cannot answer a previously-answered question.

	• Upon reaching a checkpoint, a modal appears informing the user about the checkpoint reached. • Test using this exploration that contains all interactions: https://www.oppiatestserver.org/explore/16
2.17	The user cannot: • Add an exploration to 'play later' from the (/community-library page) • Visit the learner dashboard. • Visit the creator dashboard. • Visit the preferences page
2.18	From the embedded exploration player, the user should be able to: • Play an embedded lesson in another language and change the language by returning to the introduction page. • Navigate to previously-played cards. • Learners cannot answer a previously-answered question. • Use concept cards and hints wherever provided. • View all the previous responses to any question. • Restart the exploration upon refreshing the page. • Receive a confirmation via a toast message upon completing the exploration. To run the embedded player: • Download the HTML file with the embedded player code. You can download it from this link. • Save the file in a folder on your computer. • Navigate to the folder where you saved the HTML file. • Double-click on the HTML file. It should open in your default web browser and display the embedded player.
2.19	The learner can play https://oppiatestserver.org/explore/G8ZgSj104rjE successfully.
2.20	User can: • Be interrupted by a confirmation dialog after hitting the back button in the browser.
2.23	As a logged out Learner You should be able to see two options: New to math ? - You should be able to Begin with place value Already know some math? - able to take a test. Click start test and ensure you are landing on the learning diagnostic test page . Try to exit page (before start of test, during the test , After answering a question after you done with the test) and ensure that you are back to the Oppia Classroom for Math Homepage

	When Going through the Diagnostic Test: • While giving all correct answers, ensure that there would not be any recommended lessons and you will see this message: "Test complete, well done!" • While giving some correct and some wrong answers, you should see a card with recommended lesson(s) • When answering all questions incorrectly, you should see a card with recommended lesson(s).
[2.24]	As a learner I should be able to play an embedded lesson in other language
3. Logged-in learner	
3.0	As a Learner You should be able to see two options: 1. New to math ? - You should be able to Begin with place value 2. Already know some math? - able to take a test. • When the user clicks math lesson he should be able to go back (math section) -learn/math specially in web and mobile view both - open test server in mobile to check this. SEE This https://github.com/oppia/oppia/pull/21769 for clear understanding.
3.2	After logging-in, user should be able to: • Land on the learner dashboard. However, if a user has specified a different preferred dashboard in the /preferences, they should be redirected to that selected dashboard instead. • Navigate to the learner dashboard, creator dashboard, contributor dashboard, profile page (click on the username in the dropdown), facilitator dashboard, and preferences from the profile without requiring any additional roles. .
3.9	<i>edit-profile-preferences-and-export-their-account.spec.ts</i> From the profile page, User can: • Edit avatar • See created and edited explorations.
3.12	Once an exploration is completed the learner will start the exploration from the beginning the next time they visit it.
3.13	Logged-in user (without any additional role) cannot: i) Visit the moderator page (/moderator) ii) Visit the topics and skills dashboard. (/topics-and-skills-dashboard) iii) Act as a release coordinator. (/release-coordinator) iv) Access the contributor dashboard admin. (/contributor-admin-dashboard)

	v) Act as a site admin (/admin).
3.14	The learner can: - Access the new classrooms page (at /learn): If there are 0 classrooms, they will be redirected to a 404 page; if there is 1 classroom, they will be redirected to that classroom page; and if there are more than 1 classroom, they will see all the classrooms in card format. - Access classrooms from the /community-library page.
[3.15]	"User can: - check current progress from the lesson info modal - can return to the most recent checkpoint visited and resume the exploration after the previously visited checkpoint"
[3.16]	Logged-in Learner can open Math classroom page and view lessons in it
[3.17]	Logged-in Learner can try Android App
4. Exploration Creator User Journeys	
4.1	The creator is able to see statistics about number of views, average rating, etc on the creator dashboard, and is able to view all their previously published explorations.
4.3	The creator can create an exploration with multiple states, answer groups, hints and solutions.
4.4	The creator is able to add any RTE component to the main content, answer groups, answer feedback, hints and solutions. After adding this content, the exploration should be previewable in the preview tab. Make sure the previewed content matches the content in the edit screen. Make sure to test all RTEs. RTE components include: - Images (PNG upload) - Images (SVG diagrams) - Videos: When creating an exploration on the exploration editor page, if a video is added from RTE with a start time greater than the end time it should not work. - Links - Collapsibles - Tabs - Math expressions - Concept card
4.5	The creator is able to select and add an interaction, and apply all the allowed customization args and all possible rule types. After adding the interaction, the exploration can be previewed in the preview tab. The allowed interactions are: Continue button

	2. Image region 3. Item Selection 4. Multiple choice 5. Text input 6. Drag and drop input 7. Fraction input 8. Graph input 9. Number input 10. Set input 11. Numeric expression input 12. Math equation input 13. Algebraic expression input 14. Ratio Expression Input 15. Number with units 16. Code editor 17. Pencilcode editor 18. Music notes input 19. Map input 20. End exploration
4.6	The voiceover admin is able to add a second user as a voice artist for the exploration. The voice artist should have access to add voiceovers, but not translations (text-based) in the selected language. All content components (state content, feedback, hints, solutions) should be voiceover-able. please check that you are able to play the voiceover before uploading tab navigation working correctly.(Note Ref: https://github.com/oppia/oppia/pull/21847 for tab navigation)
4.7	The creator can view the history of the exploration and revert to any old version. The creator can also view the diff between any 2 selected versions of the exploration.
4.8	The creator is able to download any version of the exploration from the history tab.
4.9	The creator can address any feedback on the explorations from the feedback tab.
4.10	[Not currently enabled] Learners' answers are logged and available for creators to study in the exploration editor's statistics tab
4.11	[Not currently enabled] Creators can not publish updates to an exploration until unresolved student answers have explicit feedback
4.12	The creator is able to open an existing exploration, make more changes and publish it. Test all the interactions and all corresponding rules as part of this. Test adding / modifying / replacing of images as well.

4.13	The creator is able to add managers, collaborators, and playtesters in the settings tab. The managers, collaborators, and playtesters are able to correctly access the exploration.
4.14	The exploration editor is able to add translations (text-based) in other languages. All content components (state content, feedback, hints, solutions) should be translatable.
[4.17]	Users can 1. Create questions from the exploration 2. Add questions from the exploration
[4.18] exploration-editor/download-any-version-exploration.spec.ts	exploration-editor/download-any-version-exploration.spec.ts Users can : 1. See history in the exploration editor 2. Can check the latest version with notes, User, and date 3. Compare any two versions from compare revision block 4. Can filter by user name 5. Can change the number of items displayed per page 6. The number of items per page can be changed by the user by using a paginator 7. User can download any changes 8. User can revert any old changes
[4.19]	User can: 1. Ask for "Help" for the editor's exploration 2. Take a tour of editor's exploration 3. Visit the help center 4. Take the translator tour
5. Curriculum Admin Journeys : Topics and skills dashboard. (These journeys require "Curriculum admin" permission)	
5.4	Within the Classroom editor page, a curriculum admin can add topics to a classroom (as well as order the topics that are taught within the classroom) and specify the dependencies between topics. The diagnostic test infrastructure should be able to use this information to find the "earliest" topic that a learner would benefit from completing. (final outcome :Editor successfully inputs the skill tree for a diagnostic test)
6. Admin User journeys (These journeys require "Admin" permission)	
6.1	All tabs on the admin page are accessible, and no errors are shown

	<i>* Note from Doc: All four tabs are tested, so this gets tested indirectly.</i>
9. Blog admin journeys (This journey requires “Blog admin” permission)	
9.1 assign-roles-to-users-and-change-tag-properties.spec.ts	<i>assign-roles-to-users-and-change-tag-properties.spec.ts</i> User can • Visit the blog editor page. • Add tag for categorizing the blog posts • Set maximum tags limit on the blog-dashboard • create and delete draft blog posts. • Appoint new blog post editors. • Remove existing blog post editors • Reduce the browser dimensions to simulate a small mobile screen and make sure there is no overlapping of buttons Note : For appointing and Removing editors access this link (with blog admin permission- https://www.oppiatestserver.org/blog-admin)
10. Voiceover Admin journeys (This Journey require Voice over admin access)	
10.1 add-voiceover-artist-to-an-exploration.spec	<i>add-voiceover-artist-to-an-exploration.spec</i> Voiceover admin can : • add voiceover artists to an exploration, and see an error when adding an invalid user.
10.2 logged-out-user/play-lesson-in-different-languages-and-listen-to-voiceovers.spec.ts	<i>logged-out-user/play-lesson-in-different-languages-and-listen-to-voiceovers.spec.ts</i> Add voice overs in English and other languages in their supported accents. Delete some voiceovers after adding and mark a few of them as stale. Voice Over Artist journey: Should be able to add/delete/mark stale the voiceovers from the exploration editor page. Adding a new voiceover will require selecting language and accent (new) from the dropdown. [prerequisite 1. Create an exploration with different cards 2. Add translations in 2 or 3 languages.]
11. Topic Manager User journeys These journey require “Topic Manager” permission, Make sure that you DO NOT have curriculum admin access	
11.1	<i>topic-manager/create-and-delete-subtopic-and-story.spec.ts</i>

topic-manager/create-and-delete-subtopic-and-story.spec.ts	The topic manager should be able to: • Access the topics and skills dashboard via the profile dropdown menu. • Make sure that keyboard focus is visible while you click any topic. • Create a subtopic and a story (add chapters to it). • Delete a story and delete subtopics and chapters
11.5 edit-and-preview-a-topic.spec.ts	<i>edit-and-preview-a-topic.spec.ts</i> The topic manager can: • Open an existing topic, change some data and save changes. • Enable or disable the practice tab to learners. • Preview lesson, practice, and revision in the topic preview.
11.7	The topic manager can: • Open an existing story, update it, and save changes. It should be previewable in the preview tab.
11.8	The topic manager can create stories with chapters in a topic and should be able to publish them only if the explorations attached to the chapters are supported on mobile. The only supported interactions are: 1. AlgebraicExpressionInput 2. Continue 3. DragAndDropSortInput 4. EndExploration 5. FractionInput 6. ImageClickInput 7. ItemSelectionInput 8. MathEquationInput 9. MultipleChoiceInput 10. NumericExpressionInput 11. NumericInput 12. NumberWithUnits 13. RatioExpressionInput 14. TextInput : (Note: • For Completing this journey tester should have task manager access only (curriculum admin access should be revoked • Tester need to create an exploration and publish that then use that exploration id when adding chapter in story • You can not add any link) 2.Topic manager does not have rights to publish the stories , admin does:
11.10	The topic manager can create questions using the questions tab in the topic editor 1. Test all supported interactions and ensure that all the rules work as expected.

11.12	<i>create-and-delete-questions-in-skill-editor.spec.ts</i> The topic manager can update an existing question, but can not publish it again. try to Make updates to all fields (including the state content, interaction, difficulty, linked skills, answer groups, hints and solution). • Test all supported interactions and ensure that all the rules work as expected. • Make sure to add / modify / replace images. And check all the buttons are working.
11.15	<i>cannot-do-curriculum-admin-actions.spec.ts</i> The topic manager should not be able to: For the topics he/she does not own) • Edit topics and stories that are not owned (can view the respective editor though).-true • Create and delete topics and skills.- • Access the classroom-admin page. • For Diagnostic test Topic manager : Can Add up to 3 skills(skills must contain 3 questions) per topic for the diagnostic test. If he/she tries to add more skills then they should not be able to continue creating another topic. Note : Topic Manager will be able to do all these tasks only for the topic he owns, so test this for both the scenarios.
13. Contributor User journeys	
13.1	The contributor can submit a question suggestion through the contributor dashboard (Include images and math expressions). USE EVERY INTERACTION. Make sure you see "Please enter feedback for the '[All other answers]' outcome." when suggesting a question. <i>This journey requires "Question submitter" permission.</i>
13.2	The contributor can submit a translation through the contributor dashboard. Test the filtering of the translations by language and by topic. Make sure to translate content with images. Use the copy tool and try out all the RTE components. The pin icon should not be visible. <i>This journey requires "Translation submitter" permission.</i>
13.3	The contributor can view their suggestion submissions while they are awaiting review. <i>This journey requires "Question submitter" and "Translation submitter" permission.</i>

13.4	The reviewer can 1. view a pending question suggestion and accept/reject it. Ensure that the page is not slow to load. 2. Check that after Accepting and rejecting a question you should be able to accept or reject next/other question(check console error also) <i>This journey requires "Question reviewer" permission.</i>
13.5	The reviewer can view a pending translation suggestion and accept/reject it (you can give yourself review rights by navigating to contributor-dashboard-admin). 1. The reviewer can filter suggestions by topic. 2. The reviewer can view all pending suggestions for a particular lesson. 3. The pin icon should be visible . 4. When accept the suggestion make sure no translation count error message showing <i>This journey requires "Translation reviewer" permission.</i>
13.6	The reviewer can view a translation in review and edit it before accepting (you can give yourself review rights by navigating to contributor-dashboard-admin). <i>This journey requires "Translation reviewer" permission.</i>
13.7	Translation submitter can see the stats when going to "Stats" tab under "Accomplishments" <i>This journey requires "Translation submitter" permission.</i>
13.8	Translation submitter can generate a certificate for contributions when going to "Stats" tab under "Accomplishments" <i>This journey requires "Translation submitter" permission.</i> Note: This journey needs a second tester or second username, as reviewer role, to approve the submissions.
13.9	Translation submitter can see already achieved badges and the badge to be unlocked when going to "Badges" tab under "Accomplishments" <i>This journey requires "Translation submitter" permission.</i> Note: This journey needs a second tester or second username, as reviewer role, to approve the submissions.
13.10	Question submitter can see the stats when going to "Stats" tab under "Accomplishments" <i>This journey requires "Question submitter" permission.</i>

13.11	Question submitter can generate a certificate for contributions when going to “Stats” tab under “Accomplishments” <i>This journey requires “ Question submitter” permission.</i> Note: This journey needs a second tester or second username, as reviewer role, to approve the submissions.
13.12	Question submitter can see already achieved badges and the badge to be unlocked when going to “Badges” tab under “Accomplishments” <i>This journey requires “ Question submitter” permission.</i> Note: This journey needs a second tester or second username, as reviewer role, to approve the submissions.
13.13	Translation reviewer can see the stats when going to “Stats” tab under “Accomplishments” <i>This journey requires “Translation reviewer” permission.</i>
13.14	Translation reviewer can see already achieved badges and the badge to be unlocked when going to “Badges” tab under “Accomplishments” <i>This journey requires “Translation reviewer” permission.</i>
13.15	Question reviewer can see the stats when going to “Stats” tab under “Accomplishments” <i>This journey requires “question reviewer” permission.</i>
13.16	Question reviewer can see already achieved badges and the badge to be unlocked when going to “Badges” tab under “Accomplishments” <i>This journey requires “Question reviewer” permission.</i>
[13.17]	Translation Reviewer can generate certificate
[13.18]	Practice Question Reviewer can generate certificate
14. Email dashboard (/emaildashboard) This journey requires “Admin” permission.	
14.1	On the email dashboard, user can: (Queries ain't working as expected for the time being. Issue)

	• Put Queries Based On Various Parameters (such as user activity, exploration creation and editing, and collection creation.) and see output with query details (including Query ID, Submitted On, Submitted By, Status, and Number of Qualified Users for the Query.). • Recheck query status to see if its status has changed from 'processing' to 'completed'.
15. Blog Editor	
[15.1]	Users can create and delete draft blog posts
[15.2]	Users unable to publish an empty blog-post, can publish a blog post (upload thumbnail image, add tags, add title and body) and delete that published blog post from the Published Tab.

Appendix B

(Coverage of E2E tests by Acceptance tests)

Color Codes Used:

 Yellow - Some part of WebdriverIO Test isn't covered by CUJs.

 Red - Very little or no part of WebdriverIO Test is covered by CUJs.

This coverage is checked by all CUJs in Appendix A (which consists of CUJs in Web QA Matrix Spreadsheet and CUJs v2 Document).

WebdriverIO Test (E2E Test)	Covered by CUJs (Collectively)
accessibility	2.16, 3.10
additionalPlayerFeatures	4.2 (The creator can play the exploration tutorial) 4.15 (The exploration editor can do the Basic setting) 2.11 - User can report exploration 3.0 - Play recommended exploration <i>Not Covered</i> should prevent going back when help card is shown - Add to 2.10 should redirect back to parent exploration correctly when parent id is given as query parameter
blog	1.15 - User should be able to do all blog page related activities
blogDashboard	9.1, [15.1], [15.2]
collections	2.9, 11.1, 11.7,
contributorAdminDashboard	13.8, 13.10 <i>Not Covered</i> should be able to filter those translation submitters, who have submitted translations between a given date range
contributorDashboard	13.2, 13.4, 7.2 should show correct featured languages
coreEditorAndPlayerFeatures	2.10, 4.3, 4.5, 4.12, 4.15 <i>Not Covered</i> should not show self-loop warning when navigating between different states should show warnings when the length of goal < 15
creatorDashboard	4.1
embedding	2.18, [2.24]

WebdriverIO Test (E2E Test)	Covered by CUJs (Collectively)
explorationFeedbackTab	2.11, 3.4, 4.9
explorationHistoryTab	4.7, 4.8
explorationImprovementsTab	● should not be present in an unpublished exploration <i>The E2E test is incomplete.</i>
explorationStatisticsTab	4.1 Check about leaving exploration from the last state isn't covered.
explorationTranslationTab	10.2, 4.6 ● should have all the state contents for voiceover in exploration language ● should have voiceover as a default mode ● should cache the selected language for translation and voiceover ● should walkthrough translation tutorial when user clicks next
extensions	2.10, 4.5 <i>Not Covered</i> ● should pass their own test suites
fileUploadExtensions	4.4
learnerFlow	2.9
navigation	1.1 to 1.13 <i>Not Covered</i> ● should visit the Fractions landing page ● should set the correct itemprop meta tags ● should set the correct og meta tags ● should set the correct application name ● should not show Dev Mode label in prod ● should report a client error to the backend
profileFeatures	3.1, 3.6, 3.7, 3.9, 2.3
profileMenuFlow	3.2, 11.1
publicationAndLibrary	4.1 ● should be correct for collaborators
skillEditor	5.1, 5.2, 11.11
subscriptionsFlow	3.6

WebdriverIO Test (E2E Test)	Covered by CUJs (Collectively)
topicAndStoryEditor	5.2, 11.1, 11.2, 11.6, 11.7, 11.9, 11.12
topicAndStoryEditorFileUploadFeatures	11.6, 11.7, 11.9
topicsAndSkillsDashboard	11.2, 11.3, 11.4, 11.13
userJourneys	3.8, <i>Not Covered</i> • should save the language selected in the footer into the preferences • should set preferred audio language selected in the Preferences page • should use language selected in the Preferences page
voiceoverUploadFeatures	10.2 <i>Not Covered</i> • should not let upload a non audio file • should not let upload a five minutes longer audio
wipeout	3.1 covers account deletion but doesn't verify following: • should delete private exploration • should set published exploration as community owned • should keep published exploration with other owner

Appendix C

(CUJ v2 Doc and Web QA Matrix Sync)

Color Codes Used:

 Yellow – The CUJ from the Web QA Matrix Spreadsheet is partially covered in the CUJ v2 document.

 Red – The CUJ from the Web QA Matrix Spreadsheet is not covered in the CUJ v2 document.

CUJs highlighted in yellow or red indicate partial or minimal coverage in the CUJ v2 doc. All such CUJs are listed in Appendix A, which ensures complete coverage of CUJs from both the document and the spreadsheet.

ID (given in case of partial, or no coverage by CUJ v2 Doc)	User	CUJ v2 Doc - CUJ	Web QA Testing - Row
	Logged-out Learner	2.11	3
	Logged-out Learner	1.12	4
	Logged-out Learner	2.13	5
	Logged-in Learner	3.3, 3.4	3
	Logged-in Learner	3.6	4
3.15	Logged-in Learner "User can: 1) check current progress from the lesson info modal 2) can return to the most recent checkpoint visited and resume the exploration after the previously visited checkpoint"	3.11 (Partially)	5
	Logged-in Learner	3.1, 3.7	6
3.16	Logged-in Learner User can open Math classroom page and view lessons in it	Not Covered	7
3.17	Logged-in Learner User can try Android App	Not Covered	8
	Anonymous Learner	2.7	3
	Anonymous Learner	2.11, 3.4	4
	Static Pages	1.2	14
	Static Pages	1.3	15

1.20	Static Pages Go to Browse the Library link	Not Covered	16
	Static Pages	1.1	17
	Static Pages	1.6	18
1.21	Static Pages <i>Covered</i> 1. Go to Privacy Policy link in the footer. 2. Scroll down and Click on 'https://www.oppia.org' link 3. Click on the back button in the browser and click on 'http://www.allaboutcookies.org/manage-cookies/index.html' 4. Click on the back button in the browser and click on ' https://www.google.com/policies/privacy/partners/ ' <i>Uncovered</i> 5. Click on the back button in the browser and click on 'https://tools.google.com/dlpage/gaoptout'	Partial tools.google.com/dlpage/gaoptout not covered	19
	Static Pages	1.18	20
	Static Pages	1.5	21
15.1	Blog Editor Users can create and delete draft blog posts	Not Covered	6
15.2	Blog Editor Users unable to publish an empty blog-post, can publish a blog post (upload thumbnail image, add tags, add title and body) and delete that published blog post from the Published Tab.	Not Covered	7
	Exploration Creator	4.6	3
4.17	Exploration Creator Users can 1. Create questions from the exploration 2. Add questions from the exploration	Not Covered	4

4.18	Exploration Creator Users can : 1. See history in the exploration editor 2. Can check the latest version with notes, User, and date 3. Compare any two versions from compare revision block 4. Can filter by user name 5. Can change the number of items displayed per page 6. The number of items per page can be changed by the user by using a paginator 7. User can download any changes 8. User can revert any old changes	Partial 4.7, 4.8	5
	Exploration Creator	4.9	7
4.19	Exploration Creator User can: 1. Ask for "Help" for the editor's exploration 2. Take a tour of editor's exploration 3. Visit the help center 4. Take the translator tour	Not Covered	8
	Exploration Creator	4.16	10
	Exploration Creator	4.5	11
	Exploration Creator	4.3	12
	Exploration Creator	4.4	13
	Exploration Creator	4.12	14
	Exploration Creator	4.1	15
	Topic Manager	11.1, 11.13	3
	Topic Manager	11.1, 11.3, 11.4, 11.13	4
	Topic Manager	11.11	5
	Voiceover Artist	10.2	3
	Curriculum Admin	5.3	4
	Embedded Lesson Player	2.18	3
2.24	Embedded Lesson Player As a learner I should be able to play an embedded lesson in other language	Not Covered	4

	Translation Submitter	13.2	3
	Translation Submitter	13.7, 13.8, 13.9	4
	Translation Submitter	13.7, 13.8, 13.9	5
	Translation Reviewer	13.5	6
13.17	Translation Reviewer	3.13 Covers Stats, Certificate Uncovered	7
	Translation Reviewer	13.14	8
	Translation Admins	7.1	9
	Translation Admins	7.2	10
	Practice Questions Submitter	13.1	11
	Practice Questions Submitter	13.1	12
	Practice Questions Submitter	13.10, 13.11, 13.12	13
	Practice Questions Submitter	13.10, 13.11, 13.12	14
	Practice Question Reviewers	13.4	15
	Practice Question Reviewers	13.4	16
13.18	Practice Question Reviewers	13.15, Certificate Uncoverd	17
	Practice Question Reviewers	13.16	18
	Practice Question Admins	7.2	19
	Practice Question Admins	7.2	20